

Part 2 – Physical Layer	4
Transmission Media	4
Guided Media	4
Unguided Media.....	5
Transmission Impairment.....	5
Transmission Methods	6
Simplex and Duplex transmission	6
Parallel vs Serial Transmission	6
Multiplexing.....	6
Line codes.....	7
Part 3 – The Data Link Layer (frames).....	8
Framing.....	8
Framing Methods	8
Byte Count	8
Start & End bytes with byte stuffing.....	8
Start & End bytes with bit stuffing	8
Flow Control.....	9
Simple Stop and Wait Protocol.....	9
Automatic Repeat Request Protocols (ARQ)	9
Stop and Wait ARQ.....	10
Piggy Backing.....	10
Go-Back N ARQ Protocol	11
Selective Repeat ARQ Protocol	11
Error Handling.....	12
Parity Bits	12
Checksums	13
Cyclic Redundancy Check (CRC)	13
Part 4 – The Data Link Layer in Broadcast Networks.....	14
Multiple Access Protocols	14
Channel Allocation problem	14
Assumptions	14
System Dependent	14

Medium Access Control Sublayer (MAC Sublayer)	15
Contention Protocols	15
Collision-free Protocols.....	16
Limited-Contention Protocols.....	16
Wireless LAN Protocols	18
Examples of Networks.....	19
Part 5 – Network Layer	20
Network Layer roles	20
Comparative Table.....	21
Internetworking.....	22
Packet Fragmentation	22
Path MTU Discovery	22
The Network Layer in the Internet	23
IPv4	23
Subnet Mask.....	25
Classless addressing	25
Public/Private IP addresses:	25
IPv6 Format	25
Other Internet Protocols	27
Routing Algorithms.....	28
Flooding.....	28
Link State Routing	28
Dijkstra's Algorithm	29
Congestion Control Algorithms	30
Leaky bucket problem	30
Token Bucket Algorithm:.....	31
Operating systems	32
The Kernel	32
Types of Systems	32
Monolythic and Microkernel Operating Systems	33
The process model.....	33
Independent Sequential Processes	34

Stack and heap	34
Process lifecycle and state	34
High Level Process Abstraction	35
Process states and transitions	35
Processes and memory and kernel:.....	36
Process Table	36
Typical System Startup	36
System Calls	36
Inter-process Communication (IPC) & Concurrency.....	38
Race Conditions and critical regions.....	38
Mutual Exclusion via disabling interrupts	39
Mutual exclusion via busy waiting.....	39
Priority Inversion Problem.....	40
Mutual exclusion without busy waiting.....	40
Scheduling	43
Scheduling algorithm goals:.....	43
Scheduling Algorithms.....	44

Part 2 – Physical Layer

The Physical layer is responsible for transmitting data to and from physical connections between devices

Transmission Media

Bits are transmitted as signals over transmission media from one node to another. These types of media are grouped into

Guided Media

Guided Media refers to data transmitted over a physical medium such as a wire

- Twisted Pair Cables
 - Used in Local Area Networks (LAN)
 - Has 4 pairs of twisted wires
 - The twisted wires are to reduce noise
 - 2 Types
 - Shielded Twisted Pairs (STP)
 - Unshielded Twisted Pairs (UTP)
- Coaxial Cable
 - Traditionally used in Ethernet Local Area Networks
 - Commonly used in Cable Television and Metropolitan Area Networks (MAN)
 - Can transmit 1Gbps over 1km
- Power Lines
 - Network using household electrical wiring
 - Convenient but unreliable
- Fibre Optics
 - Used by Internet Service Providers (ISP) to connect to homes
 - Commonly used for long distances
 - Light is reflected through glass
 - 2 Types
 - Multi-mode
 - Light bounces through a wider core
 - Uses LED light for cheaper shorter distance
 - Single-mode
 - Light travels through a narrow core without bouncing
 - Uses lasers

Unguided Media

Unguided Media refers to data transmitted via a wireless medium such as air, water, or even a vacuum

- Radio Transmission
 - o Great for long distances
 - o Can penetrate through buildings
- Microwave Transmission
 - o Used for indoor networks (WiFi)
 - o Used between communication towers and satellites (4G/5G)
 - o Signal is reflected by everyday objects
 - o Variable strength due to **Multipath Fading** (Fluctuations in the wave)
- Infrared Transmission
 - o Short range communication
 - o Unable to pass through solid objects
 - o Remote for television

Transmission Impairment

Signals can degrade and are typically affected by

- Attenuation
 - o Loss of strength over distance
- Distortion
 - o Change in signal shape or form
- Noise
 - o Unwanted signal mixing with the existing signal

Transmission Methods

Transmission is not limited to a single method. Depending on the context, there are different methods that can be used

Simplex and Duplex transmission

- Simplex
 - o One way communication
 - Input Devices (Computer Mouse, Keyboard)
- Half Duplex
 - o Two-way communication, only one device can communicate at once
 - Walkie Talkie
- Full Duplex
 - o Two-way communication, both devices can communicate simultaneously

Parallel vs Serial Transmission

- Parallel
 - o Groups of bits are transmitted simultaneously
 - o Expensive as it requires thicker cables
 - o Longer distances are unreliable as bits may desynchronise
- Serial
 - o Bits are transmitted sequentially
 - o Slower but more reliable
 - o Cheaper due to thinner cables
 - o Sending and receiving introduces an additional level of complexity

Multiplexing

Multiplexing allows multiple signals to be transmitted along a single medium. Despite the complexity, it is an efficient use of bandwidth. There are 3 major techniques to know

- Frequency Division Multiplexing (FDM)
 - o The channel is shared by placing signals at different frequencies
- Time Division Multiplexing (TDM)
 - o The channel is shared by transmitting signal within a slice of time allocated for it
- Code Division Multiple Access
 - o All signals are combined and can be decoded using a code unique for that signal

Line codes

Line codes convert data into digital signals. To ensure there are no synchronisation issues between the sender and receiver, both the sender and receiver run at the same clock speed. The clock signal can be embedded into the bit stream to encode the data

- Manchester Encoding
 - The bit is extracted whenever there is a change in direction between each clock tick
 - High-to-Low transition denotes 1
 - Low-to-High transition denotes 0
- Non-Return-to-Zero-Inverted Encoding (NRZI Encoding)
 - The bit is extracted at each clock tick
 - A Change denotes 1
 - No Change denotes 0

Part 3 – The Data Link Layer (frames)

The Data Link Layer is commonly implemented by the Network Interface Card (NIC) or Operating system (OS) and is responsible for 3 important processes

Framing

Data is transmitted through the Data Link Layer as a **Frame**. A frame can be either a block of bytes or a sequence of bits. The sender assembles a sequence of packets from the Network Layer into frame while the receiver disassembles the frames back into packets.

Framing Methods

Before a frame can be extracted, the receiver needs to identify the delimiter (Seperator), the delimiter can vary depending on the method

- If the frame is a block of bytes
 - o Byte Count
 - o Start & End Flag with Byte Stuffing
- If the frame is a sequence of bits
 - o Start & End Flag with Bit Stuffing

Byte Count

The payload is often a variable length and thus a way to denote the length of the frame is needed. This can be done by prefixing the length of the frame prior to the data. This count can sometimes be corrupted making it impossible to identify the frame boundaries

Start & End bytes with byte stuffing

At the start and end of the payload, a **flag byte** is inserted to denote the boundaries of the frame. This works fine in most cases however, if the payload contains a flag byte, then the boundary will be marked prematurely. This is known as the **Data Transparency Problem** and the solution is to insert an escape byte. This escape byte is added each time the Flag byte or the escape byte appear in the payload. This process is known as Byte Stuffing

Start & End bytes with bit stuffing

The payload can be transmitted as an arbitrary number of bits and the boundaries are denoted by a special bit pattern (0111110). To ensure the sequence doesn't end prematurely, a **0** is stuffed after each sequence of five **1**'s

Flow Control

The flow control protocols are responsible for sending frames. These protocols often send frames at a faster pace than they can be accepted, and some more complex protocols have built-in error control mechanisms to ensure all frames are delivered successfully

Simple Stop and Wait Protocol

If the network is noiseless, this protocol can be used to transmit data unidirectional. The steps are

- Sender sends a data frame
- Receiver receives a data frame
- Receiver sends an acknowledgement
- Sender receives acknowledgement
- Sender sends the next frame

There are many issues that may arise during the communication, they are but not limited to

When the below solutions are applied, you get the Stop and Wait ARQ protocol.

Problem	Solution
Damaged Frames	Implement Negative Acknowledgements for damaged frames (NAK)
Lost Frames	Set a timeout interval relative to the round-trip time. If the interval is too short, the protocol will always timeout and if the interval is too long, there will be wasted idle time
Lost Acknowledgement	Attach a sequence number to each frame so duplicate acknowledgements can be discarded

Automatic Repeat Request Protocols (ARQ)

The ARQ Protocols implement error control onto the already existing Stop and Wait protocol to allow support for noisy and unreliable networks. A Common pattern between ARQ protocols is that they retransmit frames that are lost or corrupted during transmission. This is known as Positive Acknowledgement with Retransmission (PAR)

Stop and Wait ARQ

Operations

- Sender sends a frame with a unique sequence number and waits for the receiver to acknowledge it
- If a timeout occurs, then the frame is retransmitted
- Frames with errors are rejected and a NAK is sent back
- Duplicate frames and ACKs are rejected
- The sequence numbers alternate between 0 and 1

The efficiency for this algorithm isn't ideal but can be calculated using the following formula

$$\text{Best utilization} = \frac{\text{Frame transmission time}}{\text{Frame transmission time} + \text{round trip time}}$$

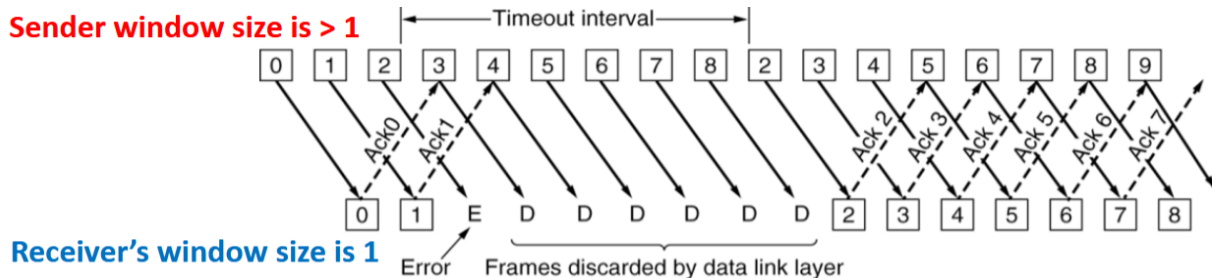
- Frame Transmission Time
 - o Frame Size (bits) / Transmission Rate (bits/s)
- Round Trip Time
 - o The time it takes to send and receive in seconds

Piggy Backing

To add extra efficiency to the Stop-and-Wait ARQ protocol, Data frames can carry acknowledgements with the data. This makes better use of the channel while reducing the computation needed from hosts.

Go-Back N ARQ Protocol

- N Frames can be sent at once before receiving an acknowledgement
- Frames are numbered sequentially up until the window size
- All frames that aren't acknowledged are buffered for the potential retransmission



The Minimum range of sequence numbers = Window Size + 1

Selective Repeat ARQ Protocol

- Allows for sending N frame before receiving an acknowledgement
- Frames are sequentially numbered up to the window size
- The receiver will accept frames within the receive window
- If there is an error, it will send a NAK

As the receiver can accept multiple frames within the window, the amount of sequence numbers needs to be increased to avoid losing a frame due to the window wrapping. To consider all edge cases, the sequence numbers available should be

$$\text{SequenceNumbers} = \text{WindowSize} * 2$$

Error Handling

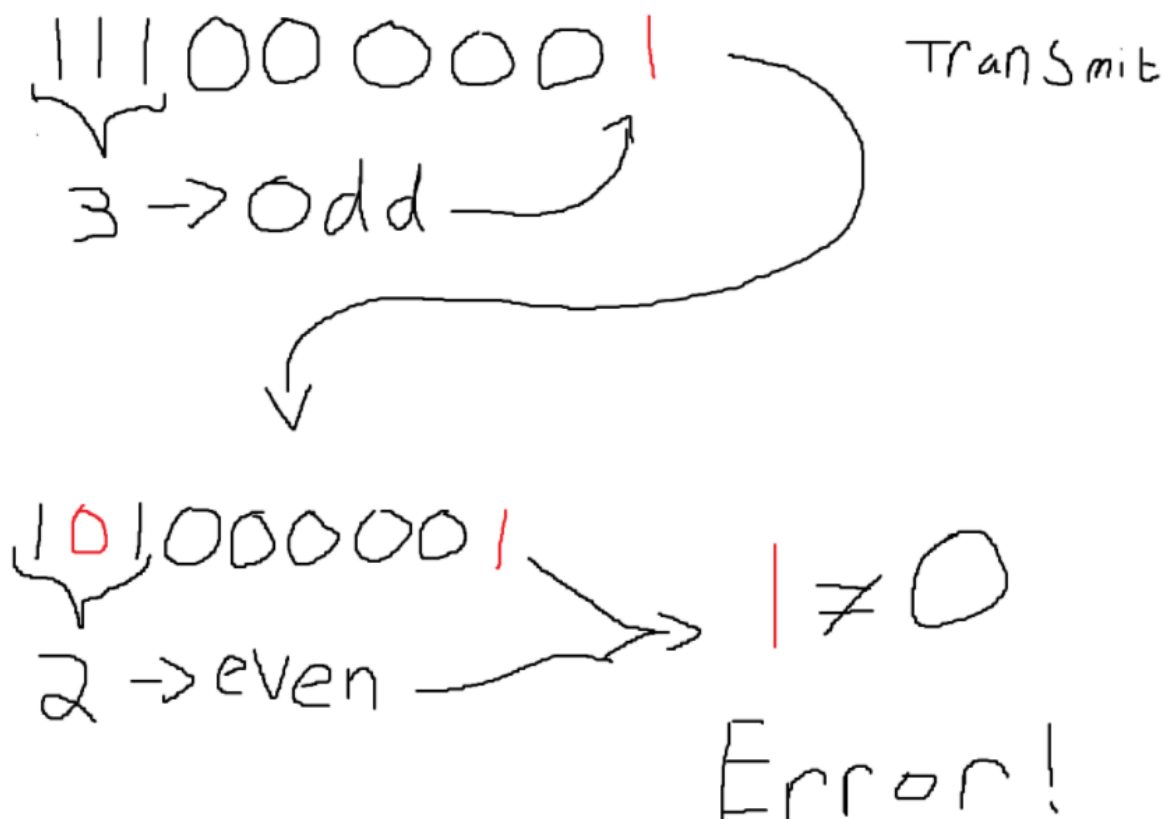
Signals weaken over long distances and if they aren't resolved, transmission errors are introduced. Thus, it is important to implement error handling strategies such as redundant data which is used both for error detection and error correction. To correct errors, you can add further redundancy but the following needs to be considered

- You must make assumptions regarding the type of error
- A missing frame will defeat any redundancy
- Error correction is used for connections likely to fail
- Must be supplemented by error detection and retransmission

Parity Bits

Redundant data can be added to the end of a byte. This bit is called a parity bit. The value here is calculated by counting all the bits in the byte and checking if the sum of these digits is an odd number or an even number. This allows the detection of an odd number of bit flips within the data.

Error Detection example with Even Parity



Checksums

Sender		Receiver	
0110011001100110 1001010101010101		0110011001100110 1001010101010101 0000010001000100	
	0110011001100110 1001010101010101 		1111111111111111
Sum	1111101110111011	Sum	1111111111111111
Complement	0000010001000100	Complement	0000000000000000 => No error

Cyclic Redundancy Check (CRC)

- Add bits to the transmitted frame (viewed as a polynomial) so that it is evenly divisible by a generator polynomial
- Sender
 - o Data has n bits
 ? 1001, n=4
 - o Choose a generator with m bits where m<n
 ? 101, m=3
 - o Add m-1 zeros to the end of the data
 ? Add 2 0s to the end of the data
 ? 1001 -> 100100
 - o Divide this number by the generator via XOR division
 - o Discard the quotient
 - o Remainder is the Cyclic Redundancy Check
 - o Subtract the Cyclic Redundancy Check from the original
 - o Send the data
- Receiver
 - o Divide the received data by the generator
 - o If the remainder isn't 0, discard the frame
 - o If the remainder is 0, there hasn't been an error

Part 4 – The Data Link Layer in Broadcast Networks

Multiple Access Protocols

Network Links can be Point-To-Point or Broadcast. In a broadcast link, a single shared transmission is shared across a broadcast channel (Multi Access Channel). Adding new hosts to this network is very easy. Some basic types of broadcast networks include

- Local Area Networks
 - o Short Delay
- Satellite Networks
 - o Larger Delay

Channel Allocation problem

- Static Channel Allocation
 - o Capacity is split among multiple competing users
 - o Bad at busy traffic
- Dynamic Channel Allocation
 - o Gives the channel when needed
 - o Uses Multiple Access Protocols

Assumptions

- All Hosts can transmit frames on the channel and receive from it
- Each frame is received by all hosts
- Hosts must fight (contend) for the use of the channel
- Frames can collide if sent simultaneously

System Dependent

- Hosts may or may not be able to detect collisions
- Timing must be assumed continuous or slotted
- A Network may or may not have carrier sensing

Medium Access Control Sublayer (MAC Sublayer)

Manages the protocols used to determine who sends on a multi-access channel. These protocols fall into 4 categories

Contention Protocols

- ALOHA
 - Used on satellite channels
 - Hosts don't listen before transmitting
 - Pure ALOHA
 - Hosts transmit whenever
 - Colliding frames are destroyed
 - If destruction is detected, a random amount of time is waited before retransmission
 - 18% efficiency under high load
 - Slotted ALOHA
 - Divides time up into discrete intervals
 - Each interval corresponds to a single frame
 - Hosts only transmit at the start of the interval
 - Requires a station to emit a "pip" signal to synchronise
 - Double efficiency compared to Pure ALOHA
- Carrier Sense Multiple Access (CSMA)
 - Improves on ALOHA by allowing hosts to listen for signals
 - Variations include
 - p-persistent
 - The p refers to the probability of transmission when the channel is idle
 - The host uses the probability to decide whether to send a frame or if it should wait
 - If the channel is idle, it transmits a frame with the probability
 - If the frame is damaged, then it waits a random amount of time before repeating
 - 1-persistent CSMA
 - An instance of p-persistent CSMA where the $p = 1$
 - Continuously listens for an idle channel
 - Sends a frame immediately
 - Non-persistent CSMA
 - Listens to the channel to check if it is idle
 - If idle, it immediately transmits

- If the channel is busy, it waits a random amount of time before listening again
- CSMA/CD
 - Using specialised electronics, the host can detect a collision when it happens.
 - If a collision is detected, the transmission is aborted and then retried after a random amount of time
 - Frame Transmission Time $\geq$ Round Trip Time
 - The time it takes to get from A to B is sometimes referred to as “Critical Time”

Collision-free Protocols

If arranged correctly, it is possible to prevent collisions completely. This is done by dividing the channel between hosts, giving each host its own private slot

- Bit-map Protocol
 - Following the frame transmission period, there is a contention period consisting of N small slots for N number of hosts on the channel
 - Hosts will insert a 1 into a slot of the contention period to mark that the slot belongs to them
 - Once this process repeats for each host, there will be no collisions as each host will wait for their turn

Some of the transmission capacity will likely be idle under light loads

Limited-Contention Protocols

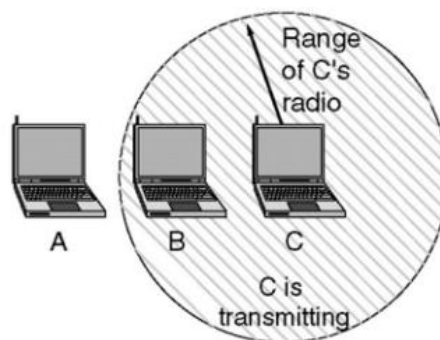
Under lighter loads, limited contention is preferable as it won't be idle unlike collision free protocols

- Adaptive Tree Walk
 - Managed like a binary tree
 - Hosts are separated into groups and can only contend for slots within their group
 - If all the contention slots in a group are taken, a subtree is created, and hosts will use those contention slots instead
- Binary Exponential Back-off
 - Time is divided into slots with a size of the worst-case Round-Trip Time
 - Every time there is a collision during the contention period, the available slots are doubled
 - After 10 collisions, the number is frozen at 1024 slots
 - After 16 Collisions, Failure is reported

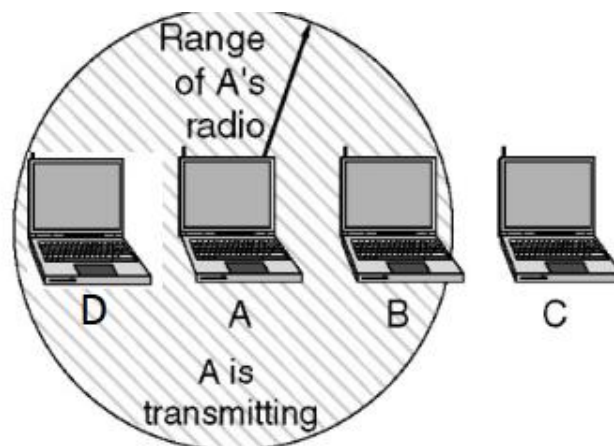
Wireless LAN Protocols

Wireless connections have various problems. For example, some stations may not be able to transmit or receive from other stations and wireless systems are unable to detect collisions as they rely on acknowledgements. Some of these problems include

- The Hidden Station Problem
 - Given stations A, B and C, some stations aren't able to detect each other
 - Despite B being able to access all stations, A and C cannot detect one another
 - If C is transmitting to B, then A doesn't know and may unknowingly transmit to B
 - In this situation, A would be ignored as B is pre-occupied with C



- The Exposed Station Problem
 - Given stations A, B, C and D
 - D and B can be detected by A
 - A and C are in range of B
 - C can only detect B
 - If A transmits to D, B will mistakenly assume there is a collision if it transmits to C
 - As there is an assumed collision, B will not transmit



- MACA Protocol
 - A wants to transmit to B
 - A sends a RTS Signal (Ready to send) to B
 - All stations in range of A recognise the RTS Signal and stays quiet until they recognise a CTS Signal (Clear to send) returned to A
 - B receives the RTS signal and sends back a CTS signal
 - All stations in range of B recognise the CTS Signal and stay quiet for the data transmission

Examples of Networks

- Ethernet
 - One shared Coaxial Cable linked all hosts together
 - The Logical Link Control (LLC) Sublayer connect all the Local Area Networks (LAN) together
 - The Medium Access Control (MAC) Sublayer manages sharing access to the physical layer
- Ethernet Bridges
 - Local Area Networks (LAN) are extended at the Data Link Layer by connecting them through bridges
 - The Local Area Network is split into segments which the bridge keeps track of in tables called **Forwarding Tables**
 - If a frame is sent and is addressed to a host on a local segment, the bridge ignores it, otherwise it will be copied to the backbone where another bridge will copy it to its own segment
- Switched Bridges
 - To overcome classic Ethernet, Hubs were created with support for twisted pairs to make adding and removing hosts easy.
 - Hubs only operate on the physical layer
 - Switches separate each host to its own port which has its own collision domain, and it operates on the Datalink Layer
- 802.11 Networks
 - Wireless Protocol
 - CSMA/CA (Collision Avoidance)
 - Station listens to the link
 - If idle, waits random amount of time to avoid collisions
 - Station inserts random backoff slots at start to minimise the chance of multiple stations sending simultaneously
 - RTS/CTS with ACK is utilised

Part 5 – Network Layer

The network layer utilises the services from the Datalink layer to route packets. It's an important layer as the technology used within the Datalink layer has a couple of bottlenecks. For example,

- Switches don't scale to large networks
- Switches can connect two LANs however, they may use different technologies
- Switches can't perform traffic control

By including the network layer, you can

- Provide independence from the Subnet technology
- Provide a uniform numbering scheme for network addresses
- Use subnets of connected routers
- Plan routes

Network Layer roles

- Internetworking
 - o Connecting two subnets with different technologies together
- Routing and Forwarding
 - o Fills routing tables and forwards packets to the correct destinations
- Congestion Control
 - o Manages data flow when too many messages are

There are two types of services offered by the network layer

- Connectionless (Datagram)
 - o Routes are not calculated in advance
 - o In a datagram subnet, each packet will have a source and destination address
 - Source referring to itself
 - Destination referring to the target
- Connection Oriented (Virtual Circuit)
 - o When a connection is established, a route from source to destination is chosen and is remembered by all routers
 - o The MPLS Protocol (Multi-protocol Labelling switching) sends packets along this path
 - o Each packet contains a number indicating order

Comparative Table

Issue	Datagram Subnet	Virtual-circuit subnet
Circuit setup	Not needed	Needed
Addressing	Each packet has a source and destination address	Each packet has a virtual circuit number
Routing	Each packet is routed independently	Route is as established by virtual circuit – all packets follow the same route
Effect of router failure	No effect aside from lost packets	All the virtual circuits that made use of it are terminated
Quality of service	Difficult	If enough resources are allocated to the circuit, easy
Congestion control	Difficult	If enough resources are allocated to the circuit, easy

Internetworking

Many networks differ in many ways such as

- Offered Services
- Protocols
- Addressing
- Packet Size
- Flow Control Mechanism

Despite this, many networks are joined together to create a larger network. This is done via internetworking which uses a standardised layer called IP (Internet Protocol).

Routers will Decapsulate and re-encapsulate packets and then use their headers to decide on the destination of the packet. Alternatively, networks can be connected through the middle of another network in a process called tunnelling which only works if there is a shared technology

Packet Fragmentation

Networks have a limit to the size of a packet, so routers need to break down packets into fragments that the network supports. If done incorrectly, this can be detrimental as the loss of a fragment results in the loss of a packet

Path MTU Discovery

Ideally, you want the biggest possible packet you can send. However, networks have different size limitations, so finding the **Maximum Transmission Unit** (MTU) is important for efficiency. This is done by

- IP Packets are sent
- Router receives the packet, and returns an error packet to denote it was too large
- The source receives the error packet and reduces the size.
- This is repeated until all the routers have been checked

The Network Layer in the Internet

The internet is a large collection of subnets. The IP Protocol facilitates the delivery of packets from one host to another through these subnets despite their differences.

IPv4

The IPv4 Header consists of 192 bits of information, it will include

- Version
- IHL
- Differentiated Services
- Length
- Identification
- DF
 - o Don't Fragment
- Offset
- Time To Live
 - o Hop count, Decrements each new router
- Protocol
- Checksum
- Source Address
- Destination Address
- Options
- Classful addressing
 - o IP addresses used to be divided into 5 classes

Class	Maximum number of hosts	Number of networks
Class A	16 million	126
Class B	65 thousand	16,384
Class C	254	2 million
Class D	Special case	Special case (multicast)
Class E	Reserved	Reserved
Broadcast 255.255.255.255		

Subnet Mask

The Subnet mask is a definition of the network and is used to identify where the data needs to be sent. They allow a single IP Address to contain multiple network which increases the limited number of IPs. Length can be any natural number between 0 and 32

- Network ID
 - o Represented by 1
- Host ID
 - o Represented by 0

If the subnet mask is 0 in length, the entire address is a host ID

Classless addressing

Classless addressing, also known as CIDR (Classless inter-domain routing) was introduced to reduce wastage.

- It is used to assign the “correct size” to a network
- Addresses are allocated in blocks called prefixes
 - o IP Address/Length

Public/Private IP addresses:

- Public Ip Address: A valid destination on the internet
 - o These are mostly exhausted
- Private Ip Address: Used within private networks
 - o Reserved prefixes are:
 - 10.0.0.0/8
 - 172.16.0.0/12
 - 192.168.0.0/16
- So that they can communicate on the internet, NAT boxes (Network address Translation) map private addresses to public ones

IPv6 Format

An IPv6 Address is a 128-bit address composed of 8 groups of 4 Hexadecimal digits. Below is the format of the header

- Version
- Traffic Class
- Flow Label
- Payload Length
- Next Header
- Hop Limit
- Source Address

- Destination Address

Other Internet Protocols

- ICMP
 - The Internet Control Message Protocol (ICMP) is used by network devices to send error messages or operational information
 - Used for router “advertisement” for finding nearby routers
- ARP
 - The Address Resolution Protocol (ARP) is used to convert an IP Address into an Ethernet Address
 - It broadcasts a request to all machines in a Local Area Network (LAN) and asks if any are using the IP Address
 - If identified, the MAC Address will be returned
 - Can be used either locally or via a router
- DHCP
 - The Dynamic Host Configuration Protocol (DHCP) can assign an IP address to a host dynamically
 - When a new host is set up
 - The DHCP Client broadcasts a DHCP discover packet
 - A DHCP relay on the Local Area Network (LAN) forwards the packet to a DHCP Server
 - The server then replies with an IP Address

Routing Algorithms

There are two phases of routing packets

- Routing Algorithm
 - o Exchanging information to decide the best possible route which is then stored in a route table
- Forwarding
 - o Routers forward the packets to the best routes denoted in the route tables

What is best in a route is decided by the following

- Topology
 - o The layout of a network
- Speed of interconnections
- Apparent load of the target host
- Congestion

Algorithms can be classified into two categories

- Adaptive Algorithms (dynamic)
 - o Changing routing decisions relative to the current situation
- Non-Adaptive Algorithms (static)
 - o Routing decisions are pre-calculated and do not change, regardless of the state of the network

Flooding

The source sends packets to every neighbour and these neighbours send the packets to their names and so on. Eventually, the destination will receive a copy of the packet. To ensure the flood doesn't go on forever

- Nodes remember which packet they've seen and don't resend if they receive it again
- Packet headers have a hop count which decrements each time the packet is sent
 - o If it reaches 0, it is thrown out

Link State Routing

Routers perform five steps to learn the network topology

- Discover: Discover neighbours and learn their addresses
- Measure: Determine the cost to each neighbour
- Construct: Create a packet with all information learned
- Distribute: Send packet to all routers
- Compute: Determine the best path based on the information

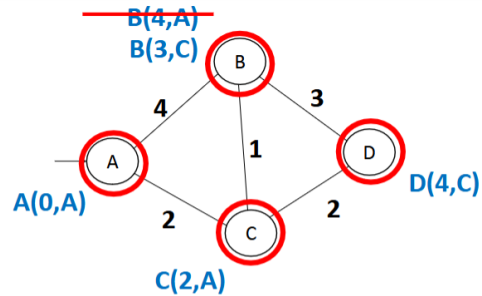
Dijkstra's Algorithm

- To begin, no paths known, each node is set to **infinity**
- The label is temporary until the shortest path is found

Example (1):

The following figure shows routers A, B, C, and D connected by links labelled with their associated costs (distances). All links are bi-directional.

Use Dijkstra's algorithm to compute the least cost routes from **node A** to all other nodes. Show your working by completing the table below with columns "**Step**" (the iteration number of the algorithm), "**Set**" (the set of permanent nodes), and "**(Cost,P)**" (current value of the cost and the predecessor node) for all X not equal to A. Enter "**INF**" to represent infinity where applicable, and enter "-" in the cells of a column after the least cost route to that node has been found.



step	set	B (Cost,P)	C (Cost,P)	D (Cost,P)
0	{A}	(4,A)	(2,A)	INF
1	{A, C}	(3,C)	-	(4,C)
2	{A, C, B}	-	-	(4,C)
3	{A, C, B, D}	-	-	-

Congestion Control Algorithms

Congestion refers to the scenario where there are too many packets present in a subnet. Congestion can be produced by

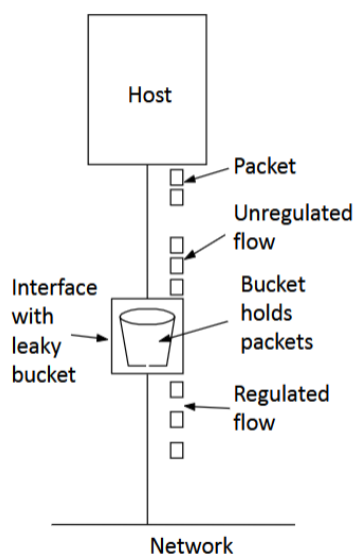
- Hardware problems
- Network topology
 - o Packets are received faster than they can be sent
 - o Not enough memory to buffer all the packets

There are various solutions to either prevent or mitigate congestion

- Network Provisioning
 - o Make network big enough to handle the traffic
- Traffic Aware Routing
 - o Compute Routes with traffic in mind
 - o Shift away from busy routes
- Admission Control
 - o Only accept packets if the network has the capacity to handle it
- Traffic Throttling
 - o Sends a choke packet to other hosts to tell them to slow down
- Load Shedding
 - o Dropping packets to reduce load

Leaky bucket problem

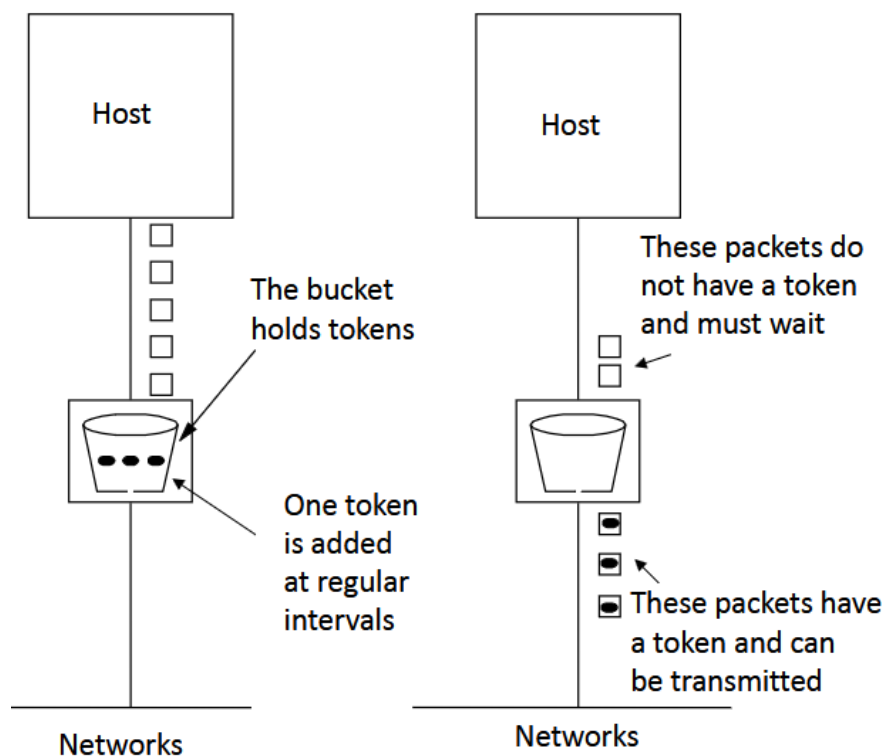
- Turns an uneven flow of traffic into an even flow
- Enforces a rigid output pattern, it doesn't matter how bursty how bursty the traffic is



- For the many applications, it's better to allow some variation in speed, a more flexible algorithm is needed

Token Bucket Algorithm:

- Allows for output variation based on burst size
- The bucket holds tokens
 - o Tokens are generated by a clock
 - o To be transmitted, a packet must capture and destroy a token
- This algorithm allows idle hosts to save permissions up to the maximum size of the bucket (N). Bursts of up to N packets can be sent at once
 - o This allows some burstiness into the output stream, and gives faster responses to sudden bursts
- The potential problem with this is that it once again allows large bursts
 - o The way to counter this is to put a leaky bucket after a token bucket



Operating systems

An operating system is an abstraction of the computer hardware. It manages system resources such as

- Process Management
 - o Allows multiple programs to run at the same time
- Device and Memory Management
 - o Manages/protects memory and other devices
- Resources are shared in space and time

The Kernel

The Kernel is the core of the operating system, and it controls access to the hardware and system resources. It provides a bridge/interface between the hardware and higher layers. This access can be given by activating a privileged kernel mode which has operations for hardware and will have system wide consequences.

- A system call will cause User Level Processes to switch to kernel mode
- Operations within the memory allocated to a process will not require kernel mode privileges.

Types of Systems

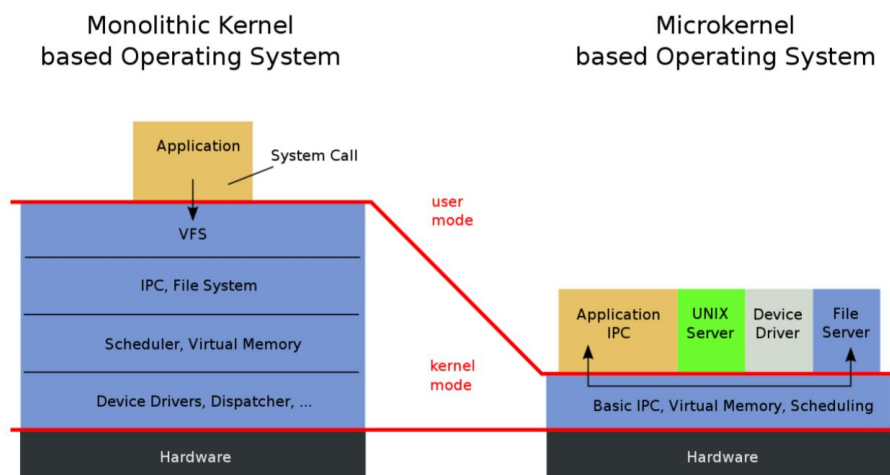
Computer systems can include

- Mainframes
- Servers
- Personal Computers
- Mobile Devices

Each of these systems will have their own different requirements

Monolythic and Microkernel Operating Systems

- Monolythic System
 - o The operating system is essentially a giant library that all applications link too
 - o All operations within this type of operating system have system wide consequences, even if unrelated
 - o For an oddly specific example (Fareed definitely experienced this)
 - A bug in an audio driver accessed by a media player can crash the entire system including unrelated applications such as a database client
- Microkernel System
 - o The kernel has the bare minimum implemented with the intention of improving reliability



The process model

Everything is built upon the process model. A process is a program that is currently being executed.

- Each process has its own address space
- Processes can run the same program text
- Processes can create children processes
- 2 distinct processes can execute the same program

Independent Sequential Processes

Every process has its own virtual

- Program counter
- Registers
- Variables
- Flow of control

To run a process

- Copy values from virtual program counter to the physical program counter

To Switch processes

- Copy the physical program counter to the virtual program counter
- Copy the value from the next processes program counter to the physical program counter

Only one process can be active however, the CPU switches between processes and it maintains the state from before it was suspended

Stack and heap

- The stack is automatically managed by the operating system. The usage is known at compile time.
 - o Typically, much smaller than the heap
- The heap is dynamic memory, and its state can't be known at compile time. It is a bit more costly and error prone with the benefits of being powerful, the programmer will need to request memory allocation and deallocation

Process lifecycle and state

Processes have a lifecycle which is management. There are three stages to the lifecycle

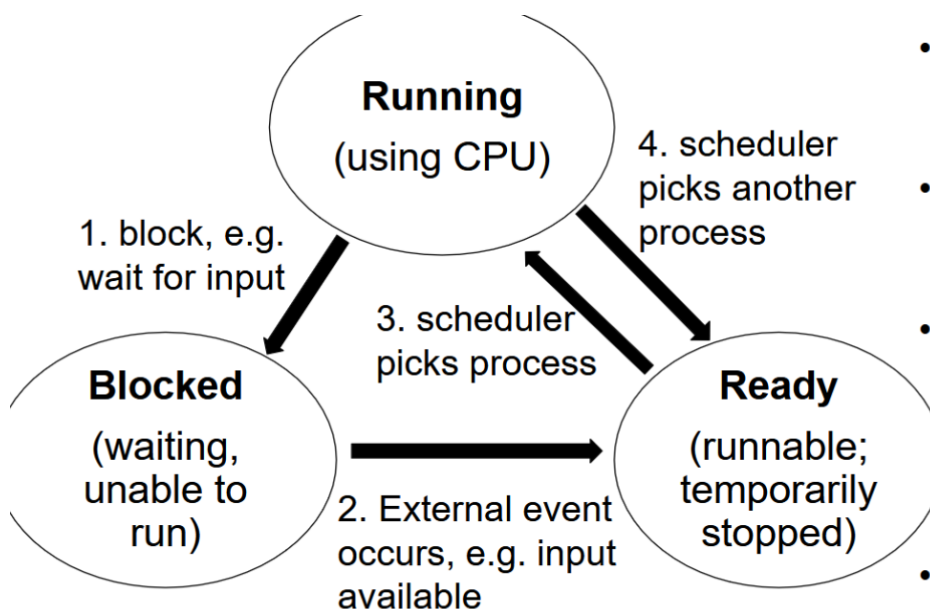
- Process initialisation
 - o Program startup
- Process execution
- Process termination
 - o Voluntary or involuntary exit

Termination condition	Type of termination
Normal exit	Voluntary
Error exit	Voluntary
Fatal error	Involuntary
Killed by another process	Involuntary

High Level Process Abstraction

The entire system can be viewed as a set of processes. Processes can create and kill child processes. These processes need to communicate between each other

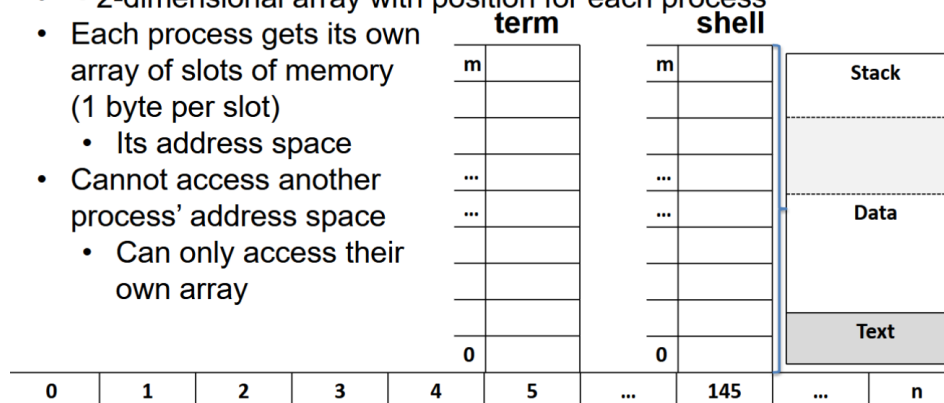
Process states and transitions



- Transition 1: OS detects that the process cannot continue
- Transition 2: external event occurs
- Transition 3&4: caused by the scheduler
- The scheduler needs process information

Processes and memory and kernel:

- ~ 2-dimensional array with position for each process
- Each process gets its own array of slots of memory (1 byte per slot)
 - Its address space
- Cannot access another process' address space
 - Can only access their own array



- The kernel accesses whole 2-dimensional array (privileged access)
 - **The OS maintains a process table that represents this model**

20

Process Table

Typical process management fields	Registers	Program counter	Program status word
	Stack pointer	Process state	Priority
	Scheduling parameters	Process ID	Parent process
	Process group	Signals	Time started
	CPU time used	Children's CPU time	Time of next alarm
Memory management	Pointer to text segment info	Pointer to data segment info	Pointer to stack segment info
File management	Root directory	Working dir.	File descriptors
	User ID	Group ID	

Typical System Startup

A Bootstrap program is loaded from the first section of the active disk partition. This program loads the boot program. The boot program looks for the boot image which is typically the operating system. Individual parts of the boot image is loaded into specific memory locations. Next the kernel starts to initialise processes. Once all the kernel setup is completed, application-level processes such as the shell and user interface are initialised to make the system usable

System Calls

To allow applications to manage processes, the operating system such as POSIX defines a standard API to interface with process management, file systems, etc...

Call / procedure	Description
pid = fork()	Create a child process identical to parent
s = execve(name, argv, environp)	Replace a process' core image
pid = waitpid(pid, &stat, options)	Wait for a child to terminate
exit(status)	Terminate process execution and return status

Inter-process Communication (IPC) & Concurrency

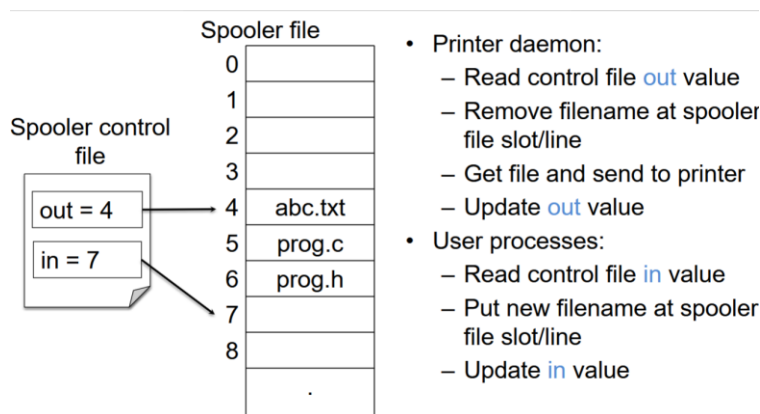
Processes need to communicate with each other to form an overall task however, system integrity needs to be maintained. There are three main problems to consider

- How to share the information
- How to prevent processes interfering with each other
- How to ensure correct sequencing

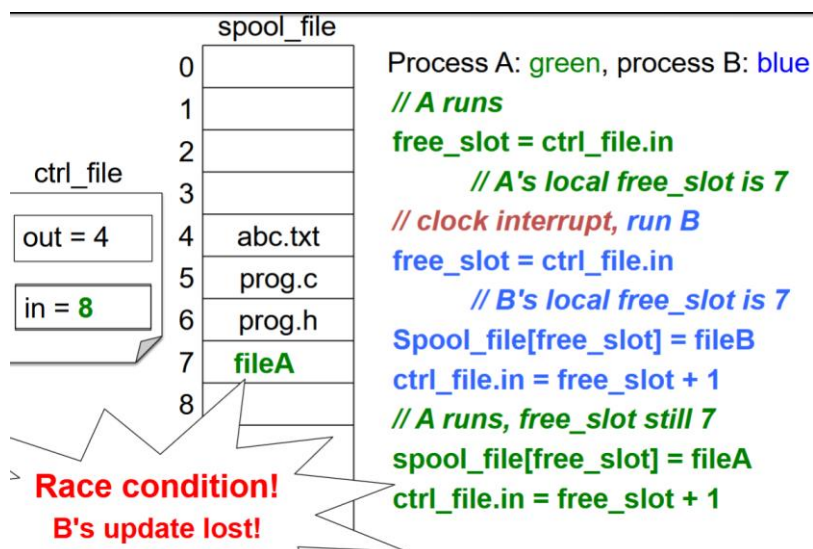
Race Conditions and critical regions

Processes communicate by sharing a common storage. For example, a printer service

- User processes add filenames to a shared print spooler file
- The printer checks this regularly and removes a file from the list while fetching it
- The file is printed

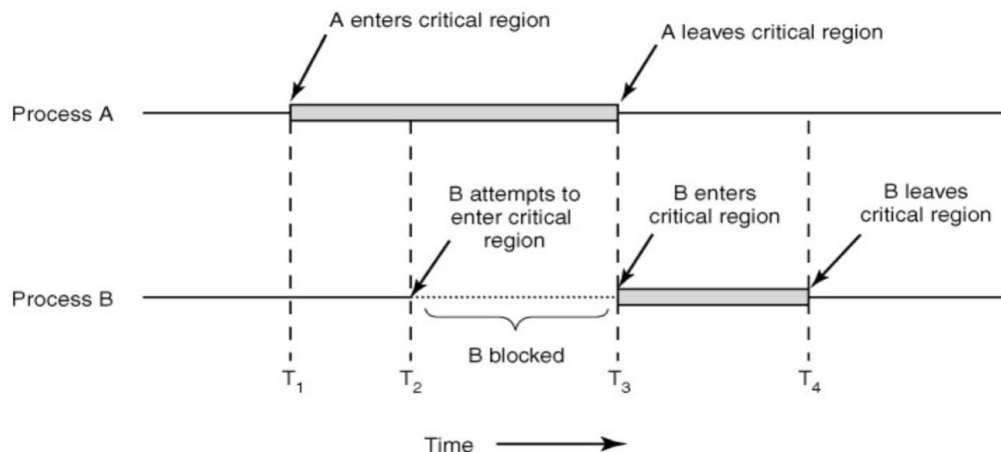


Sometimes if two files are sent to the file very close to each other, a race condition will occur and a memory location will be affected twice, in this case, the location of the file is overwritten causing file B to be lost



The solution to this race condition would be mutual exclusion for critical regions – Preventing more than one process accessing shared data at the same time.

- There are four conditions for correct and efficient data sharing
 - o No 2 processes should be in a critical region simultaneously
 - o Do not assume the number or the speed of CPUs
 - o No process outside a critical region can block another process
 - o No process can wait forever to enter its critical region



Mutual Exclusion via disabling interrupts

All interrupts are disabled upon entering a critical region. This means clock interrupts are also disabled which then stops other processes from being able to be scheduled

Mutual exclusion via busy waiting

Definition: Busy waiting – A loop doing nothing

- Approach 1: Lock Variables
 - o There is a shared lock variable is checked whenever a process needs to enter a critical region.
 - o If the lock is free, the process locks it and enters the critical region
 - o If it is locked, it waits
 - o When the process is finished, the lock is freed
 - o The race condition is moved to the lock rather than the critical region
- Approach 2: Strict Alternation

```

// process 0 code
while True:
    while turn != 0:
        do_nothing
    critical_region()
    turn = 1 // alternate
    noncritical_reg()

// process 1 code
while True:
    while turn != 1:
        do_nothing
    critical_region()
    turn = 0 // alternate noncritic
    ()

```

- Approach 3: Peterson's Solution

- Functions such as **enter_region** and **leave_region** define access to the critical region

- Two processes with ids: 0 and 1
- Global variables: **turn** and **interested array** of 2 elements to indicate process interest (1 indicates interest, initialised to {0, 0})

```

enter_region(my_id):
    other_id = 1 - my_id
    interested[my_id] = 1
    turn = my_id
    while turn == my_id
        and interested[other_id] == 1:
            do_nothing

```

```

leave_region(int my_id)
    interested[my_id] = 0

```

17

Priority Inversion Problem

- Consider two processes:
 - H with high priority and L with low priority
 - Scheduling rule: run H whenever ready
- 1. H is blocked on I/O outside critical region (cannot run)
- 2. L is in its critical region**
- 3. I/O completes, H becomes ready
- 4. H runs, L is put on ready queue (not running)
- 5. H begins busy waiting for L to leave critical region
- but L is never rescheduled because H is busy on processor doing nothing**

19

Mutual exclusion without busy waiting

The use of Inter-process communication primitives until it's safe to enter the critical region.

- The producer-consumer (bounded buffer)
 - Two processes shared a fixed sized buffer
 - The producer puts items in and the consumer takes them out
 - If the buffer is full, the producer can't produce
 - If the buffer is empty, the consumer can't consume
- Lost wakeups

- Assume an empty buffer, consumer reading count as 0
- Scheduler pre-empted consumer and runs the producer
- Producer adds item, increments count and wakes up consumer
- Consumer runs again, checks count, calls sleep
- Producer adds items until buffer is full
- Both processes are sleeping and will never wake up

- Dijkstra's Semaphore Solution to lost wakes
 - o Two operations
 - Down
 - If value is larger than 0, decrement
 - else sleep
 - Up
 - Increment
 - Wake up process if sleeping
 - o Down can block, Up can't block
 - o Semaphores are shared memory
- Using Semaphores to solve the Producer-Consumer issues
 - o Using 3 Semaphores can solve it
 - Full
 - o Full Slots
 - Empty
 - o Empty Slots
 - Mutex
 - o Buffer guard for mutual exclusion

N = 10
 sem mutex = 1
 sem empty = N
 sem full = 0

producer: while True: item = produceltem() down(empty) down(mutex) insertItem(item) up(mutex) up(full)	consumer: while True: down(full) down(mutex) item = removetem() up(mutex) up(empty) consumetem()
--	--

- Semaphore problems
 - o Error Prone
 - o Can deadlock if not ordered correctly
- Mutex: Binary Semaphore
- Monitors: In simple terms, a wrapper for a semaphore
- Implementing Semaphores
 - o Actions need to be atomic
 - o Up and Down needs to be implemented via system calls
 - OS should disable interrupts
 - Test semaphore value, updates it and puts to sleep if necessary
 - OS enables interrupts

Scheduling

Scheduling is required when

- A Process exists
- A process blocks I/O
- A process is waiting for a resource

It is needed when

- A new Process is created
- An I/O operation occurs
- A clock interrupt occurs
 - o Decide if the process has been running for long enough
 - o Non-preemptive scheduling
 - Pick process
 - Let it run until it blocks or gives up to processor
 - o Preemptive scheduling
 - Pick process
 - Let it run for maximum time interval

Scheduling algorithm goals:

All Systems	Batch Systems	Interactive Systems	Real-time systems
- Fairness - Policy enforcement (priority should be given to critical processes) - Balance (Keep all parts of the system busy)	- Maximise throughput - Minimise turnaround (wait time for output – average) - Maximise utilisation	- Optimise response time - Proportionality (measure user expectations of task length)	- Meet deadlines - Avoid Losing time-critical data - Predictability (Avoid degradation of data quality)

Scheduling Algorithms

- First come, First Served
 - o First job to arrive goes first
- Shortest Job First
 - o Shortest job goes next
- Shortest Remaining time
 - o After each job or arrival, the job with the shortest remaining time goes next
- Round Robin
 - o Each process is assigned an interval called a **quantum**
 - o If the task is still running at the end of the quantum, give the CPU another process and push the old process back to the end of the list
- Priority
 - o Give the CPU the task with the highest priority
 - o Problems
 - Can run indefinitely
 - Will idly consume memory if I/O isn't prioritised
 - o Priority queue
 - Higher priority runs before lower priority
 - Round Robin scheduling

Policy versus mechanism

- Have assumed processes belong to different users
 - Scheduler's job is to ensure fairness etc.
- Consider DBMS process with children working on different queries or specific functions
 - Parent process may have good idea of priorities among children
- **Solution:** separate scheduling mechanism from scheduling policy – allow parameterisation of scheduling by user processes
 - E.g. kernel provides call for parent processes to set priorities among children
 - Mechanism provided by kernel, policy determined by user process